福州大学

《Linux 操作系统设计实践》实验报告

学号：102401339　姓名：任宝硕　专业：计算机科学与技术　班级：3 班

实验一　进程管理

实验环境

本实验在 macOS 27.0（Darwin 27.0.0，arm64）环境下完成，使用 Apple clang 编译器辅以 GNU Make 3.81 通过在 Otty 终端上运行的 zsh 编译运行。

实验内容

编写了一个父子进程管理演示程序，由 main.c、child.c、child.h、hello.c 和 Makefile 五个文件组成。main.c 为主程序，负责注册信号处理函数和调用创建子进程的函数；child.c 实现子进程的创建、程序替换和等待，child.h 声明 create_child()函数；hello.c 为子进程通过 execl()执行的目标程序。Makefile 将 main.c 和 child.c 编译、链接为 main，将 hello.c 编译为 hello，并提供 clean 目标清理可执行文件和目标文件。

主程序启动后调用 signal()注册 SIGINT 信号处理函数 on_sigint()。主程序收到 SIGINT 时，处理函数输出信号名称和编号。终端中的 Ctrl+C 可以触发这个信号。

主程序调用 create_child()，由 fork()创建子进程，并根据返回值进入父进程或子进程分支。创建失败时，程序通过 perror()输出错误信息，并以退出码 1 结束。子进程调用 execl("./hello", "hello", NULL)，将当前执行的程序替换为 hello。hello 输出“这是被 exec 加载的 hello 程序”，随后返回 0。若 execl()失败，子进程输出错误信息，并以退出码 1 结束。

父进程调用 wait(NULL)等待并回收子进程，不保存子进程的退出状态。wait()返回后，父进程输出子进程的进程号和结束提示，返回主函数，输出“程序结束”。

程序代码

main.c

```c
#include <signal.h>
#include <stdio.h>

#include "child.h"

/* SIGINT 处理函数 */
void on_sigint(int sig) {
  printf("\n 捕获到信号 SIGINT(%d)\n", sig);
}

/* 主函数：注册信号处理，创建子进程 */
int main(void) {
  signal(SIGINT, on_sigint);

  create_child();

  printf("程序结束\n");
  return 0;
}
```

hello.c

```c
#include <stdio.h>

/* 被 exec 加载的子程序 */
int main(void) {
  printf("这是被 exec 加载的 hello 程序\n");
  return 0;
}
```

child.h

```c
#ifndef CHILD_H
#define CHILD_H

void create_child(void);

#endif
```

child.c

```c
#include "child.h"

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#include <sys/wait.h>

/* fork 子进程，子进程 exec 加载 hello，父进程等待 */
void create_child(void) {
  pid_t pid = fork();
  if (pid < 0) {
    perror("fork");
    exit(1);
  }

  if (pid == 0) {
    execl("./hello", "hello", NULL);
    perror("exec");
    exit(1);
  }

  wait(NULL);
  printf("子进程 %d 已结束\n", pid);
}
```

Makefile

```
all: main hello

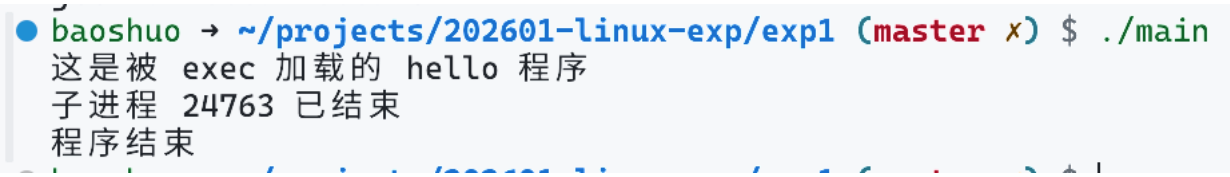
main: main.o child.o
    gcc -o main main.o child.o

hello: hello.c
    gcc -o hello hello.c

clean:
    rm -f main hello *.o

.PHONY: all clean
```

运行截图



运行./main 后，子进程通过 execl()加载 hello 程序，输出“这是被 exec 加载的 hello 程序”。父进程等待并回收子进程后，输出子进程号 24763 和结束提示，随后输出“程序结束”。

实验总结

1. 实验程序分为 main.c、child.c 和 hello.c 三个源文件，由 child.h 声明函数接口，Makefile 管理编译。主程序注册 SIGINT 信号处理函数，再调用 create_child()创建子进程。子进程通过 execl()执行 hello，父进程通过 wait(NULL)等待并回收子进程。运行截图中，hello 先输出提示信息，父进程随后输出“子进程 24763 已结束”和“程序结束”，与代码中的执行流程一致。
2. 实验中，我学习了 Makefile 中目标、依赖和命令的关系。main 依赖 main.o 和 child.o，make 使用内置的隐式规则将对应的.c 文件编译为.o 文件，再执行链接命令生成 main。hello 由 hello.c 编译生成，不经过单独生成目标文件的步骤。修改源文件后，make 根据文件的修改时间判断哪些目标需要重新构建。clean 目标用于删除编译结果，.PHONY 将 all 和 clean 声明为伪目标，避免同名文件影响规则执行。
3. 学习 fork()时，我理解了它在父子进程中返回不同值的用法。父进程得到子进程的 PID，子进程得到 0，返回-1 表示创建失败。child.c 根据返回值安排执行分支：子进程加载 hello，父进程等待子进程结束。父子进程的运行顺序由系统调度，代码中的 wait()使父进程在正常等待到子进程结束后再输出结束提示。
4. execl()用目标程序替换当前进程执行的程序，不创建新进程。本程序中的第一个参数是目标路径./hello，第二个参数是新程序的 argv[0]，NULL 表示参数列表结束。execl()成功后不返回，子进程从 hello 的入口开始执行；失败时才会执行后面的 perror()和 exit(1)。父进程使用 wait(NULL)回收子进程，但没有保存退出状态，因此截图中的结束提示不能用来判断子进程的退出码。

5. 信号处理部分使用 signal()注册 SIGINT 的处理函数，主程序收到信号后输出信号名称和编号。这个信号可由终端中的 Ctrl+C 触发，与子进程退出时产生的 SIGCHLD 不同。当前截图展示了程序执行和子进程回收的过程，没有展示 SIGINT 的处理结果。代码也没有检查 wait()的返回值或处理等待被信号中断的情况，这部分需要另行验证。